

Branches & Merging

Working safely with Git — parallel timelines for your code

Welcome back, Shoaib! Think of Git branches like parallel dimensions or different save slots in a video game. Right now your code lives on the main line called the **main** branch. When developers want to add a new feature or test something without breaking their working code, they create a new branch, work on it safely, and then merge it back into main when it's done!

1 What is a Git Branch?

main	Your main production-ready code.
Feature Branch	e.g. <code>feature-login</code> or <code>practice-branch</code> — a copy of main where you can make changes safely.
Merging	Combining the changes from your feature branch back into main.

2 Essential Branch Commands

Action	Command
Check current branch	<code>git branch</code>
Create a new branch	<code>git branch <branch-name></code>
Switch to a branch	<code>git checkout <branch-name></code>
Create AND switch (fast way)	<code>git checkout -b <branch-name></code>
Merge a branch into main	<code>git merge <branch-name></code> (must be on main first)
Delete a branch	<code>git branch -d <branch-name></code>

3 Today's Hands-On Exercise 🌟

Let's practice this live in your python learning folder!

Step 1 — Create and Switch to a New Branch

Open your terminal in VS Code and run:

```
git checkout -b practice-branch
```

(This creates a new branch named practice-branch and immediately switches you to it.)

Step 2 — Verify Your Branch

Run:

```
git branch
```

You should see an asterisk * next to practice-branch, showing you are currently on that branch!

Step 3 — Make a Test Change

Create a new simple Python file in your folder called `branch_test.py`. Add a line of code to it, like:

```
print("Testing Git branches!")
```

Stage and commit it on your new branch:

```
git add .
git commit -m "Added branch_test.py on practice-branch"
```

Shoaib's Result

```
PS C:\Users\shoaib\Desktop\python learning> git commit -m "added day59.py on practice-branch"
[practice-branch 3eac882] added day59.py on practice-branch
1 file changed, 1 insertion(+)
create mode 100644 backend/day59.py
```

✔ Spot on!

Your commit on practice-branch was successful. Notice how Git printed **[practice-branch 3eac882]** — that confirms your changes were saved specifically to practice-branch, while your main branch remains untouched.

Step 4 — Switch Back to main and Merge

Now let's complete the cycle by bringing your changes from practice-branch into your main branch! Run these commands in your VS Code terminal one by one:

```
1. git checkout main
2. git merge practice-branch
3. git branch -d practice-branch
```

Shoaib's Result

```
PS C:\Users\shoaib\Desktop\python learning> git branch -d practice-branch
Deleted branch practice-branch (was 3eac882).
```

🎉 Outstanding job!

You just completed the full Git branching cycle.

What You Accomplished

1. Created a new feature branch (practice-branch).
2. Made changes and committed them safely.
3. Switched back to main and merged those changes.
4. Deleted the temporary branch to keep your project clean.

🔑 Key Takeaway for Day 59

Whenever you work on a real-world project or with a team:

- Never code directly on main when working on a big feature.
- Always create a new branch (`git checkout -b <name>`), do your work, test it, and then merge it back into main!

Day 59 is complete! ✔

Up next in your roadmap →